UD00 part 2

4.1 PREGUNTA ENTREVISTA 01 1
4.2 PREGUNTA ENTREVISTA 02 2
4.3 PREGUNTA ENTREVISTA 03 5
4.4 PREGUNTA ENTREVISTA 04 6
4.5 PREGUNTA ENTREVISTA 05 7
4.6 PREGUNTA ENTREVISTA 06 16
4.7 PREGUNTA ENTREVISTA 07 25
4.8 PREGUNTA ENTREVISTA 08 29
4.9 PREGUNTA ENTREVISTA 09 30
4.10 PREGUNTA ENTREVISTA 10 34
4.11 PREGUNTA ENTREVISTA 11 36
4.12 PREGUNTA ENTREVISTA 12 39
4.13 PREGUNTA ENTREVISTA 13 40
4.14 PREGUNTA ENTREVISTA 14 42
4.15 PREGUNTA ENTREVISTA 15 45

4.1 PREGUNTA ENTREVISTA 01

Explica i posa un exemple senzill d'una funció lambda en Python 3.
Una función lambda es una función pequeña y anónima que se define en una sola línea.
Su estructura básica es la siguiente:
-Sin nombre: Son anónimas, aunque se pueden guardar en una variable.
-Una sola expresión: Sólo pueden contener una operación cuyo resultado se devuelve de forma automática.
-No lleva return

```
doble = lambda x: x * 2
print(doble(5))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-2.1.py
10
```

```
suma = lambda a, b: a + b
print(suma(3, 5))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-2.1.py
8
```

Després, posa un exemple que utilitze les funcions de Python:
Utilitzant "map()" (i "list()" e "int()" com a recolzament per convertir a llista i a nombres sencers) heu de llegir de teclat una cadena de text formada per números separats amb espai i transformar-la en una llista formada per nombres sencers.

```
cadena = input("Introduce números separados por espacios: ")
numeros = list(map(int, cadena.split()))
```

```
print(numeros)
```

-list nos permite convertir el resultado de map en una lista.
-map(int, ...) convierte cada elemento de texto en int.
-cadena.split( ) separa el texto por espacios

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-2.1.py
Introduce números separados por espacios: 5 8 10 9
[5, 8, 10, 9]
```

Utilitzant "filter()", elimina de la cadena anterior els números menors que 10.
Amb la cadena resultant i utilitzant "reduce()", retorna la multiplicació dels elements de la llista.

```
from functools import reduce

cadena = input("Introduce números separados por espacios: ")
numeros = list(map(int, cadena.split()))
print(numeros)

def mayor_igual_10(x):
    return x >= 10

numeros = list(filter(mayor_igual_10, numeros))
print(numeros)

resultado = reduce(lambda x, y: x * y, numeros)
print("El resultado de la multiplicación es= ",resultado)
```

-La condición x >= 10 usada con el filter hace que solo pasen los números que sean 10 o mayores.
-Luego utilizamos la función junto al reduce para que devuelva la multiplicación de los elementos de la lista

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-2.1.py
Introduce números separados por espacios: 5 2 3 10 20 4
[5, 2, 3, 10, 20, 4]
[10, 20]
El resultado de la multiplicación es=  200
```

4.2 PREGUNTA ENTREVISTA 02

Realitza una aplicació que comprove si una solució d'un Sudoku és correcta o no. L'explicació dels regles del Sudoku els teniu ací https://es.wikipedia.org/wiki/sudoku
El Sudoku serà llegit d'un fitxer "Sudoku.in", format per 9 línies on cadascuna d'aquestes estarà formada per 9 números separats per espais. El programa ens mostrarà per consola i el resultat serà mostrat per consola si és correcte o no.
Internament, el programa realitzarà la comprovació de si el Sudoku és o no correcte en una funció definida com a "def esSudokuCorrecto(miArrayBi)" que retornarà "true" si és correcte, "false" en cas contrari.

```
def esSudokuCorrecto(miArrayBi):
```

```
    # Comprobar filas
    # Con set eliminamos los números repetidos, luego comprobamos si siguen habiendo 9, en caso de que si, es que no hay números repetidos
    for fila in miArrayBi:
        if len(set(fila)) != 9:
            return False

    # Comprobar columnas
    # Aqui con el doble for recorremos todas las filas de cada columna y usaremos el set como anteriormente para ver que no hayan números repetidos
    for columna in range(9):
        columna_actual = []

        for fila in range(9):
            columna_actual.append(miArrayBi[fila][columna])

        if len(set(columna_actual)) != 9:
            return False

    # Comprobar cuadrados 3x3
   # Aquí con el primer doble for seleccionaremos cuadrado por cuadrado (3x3), ya que cogeremos tanto filas y columnas de 3 en 3.
Creamos cuadrado=[] para almacenar y comprobar los números de estos.
Una vez seleccionado el primer cuadrado usamos el siguiente doble for para introducir en la lista cuadrado[] (con .append) cada número del 3x3. Por último usaremos el set como anteriormente para ver que no hayan números repetidos y comprobaremos su longitud.
En caso de que todos los cuadrados sean correctos pasaremos finalmente al return True

    for fila in range(0, 9, 3):
        for columna in range(0, 9, 3):

            cuadrado = []

            for i in range(fila, fila + 3):
                for j in range(columna, columna + 3):
                    cuadrado.append(miArrayBi[i][j])

            if len(set(cuadrado)) != 9:
                return False
```

```
    return True

# Aqui guardaremos el Sudoku a comprobar
miArrayBi = []

#Abrimos el archivo Sudoku.in en modo lectura con "r"
with open("Sudoku.in", "r") as archivo:
#Con el for recorremos cada línea, los separamos con el split y los pasamos a int para poder comprobarlos con map(int, ) y guardamos el Sudoku en la lista miArrayBi para poder comprobarla
    for linea in archivo:
        numeros = list(map(int, linea.split()))
        miArrayBi.append(numeros)


# Comprobar Sudoku
if esSudokuCorrecto(miArrayBi):
    print("Correcte")
else:
    print("Incorrecte")
```

En este caso podemos ver como el Sudoku es correcto:

Sudoku.in

```
1   5 3 4 6 7 8 9 1 2
2   6 7 2 1 9 5 3 4 8
3   1 9 8 3 4 2 5 6 7
4   8 5 9 7 6 1 4 2 3
5   4 2 6 8 5 3 7 9 1
6   7 1 3 9 2 4 8 5 6
7   9 6 1 5 3 7 2 8 4
8   2 8 7 4 1 9 6 3 5
9   3 4 5 2 8 6 1 7 9
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-sudoku.py
Correcte
```

Y aquí podemos ver como el Sudoku es incorrecto:

Sudoku.in

```
1   1 2 3 4 5 6 7 8 9
2   1 2 3 4 5 6 7 8 9
3   1 2 3 4 5 6 7 8 9
4   1 2 3 4 5 6 7 8 9
5   1 2 3 4 5 6 7 8 9
6   1 2 3 4 5 6 7 8 9
7   1 2 3 4 5 6 7 8 9
8   1 2 3 4 5 6 7 8 9
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00-sudoku.py
Incorrecte
```

4.3 PREGUNTA ENTREVISTA 03

Realitza una aplicació que sol·licite una cadena de text. El programa ha de dir quantes vegades ocorre cadascun d'aquests patrons sense distingir majúscules i minúscules: "00" "101", "ABC", "HO". Un caràcter pot formar part de més d'un patró oposat. Per exemple: En la cadena "000" el patró "00" apareix dues vegades (una comença en la posició 0 i una altra comença en la posició 1). Internament, el programa realitzarà el compte de patrons amb una funció definida com "def numeroPatrones(text)" que retornarà un nombre enter amb el nombre de patrons oposats.

```
def numeroPatrones(text):
    text = text.upper()
    contador = 0

    for i in range(len(text)):
        if text[i:i+2] == "00":
            contador += 1
        if text[i:i+3] == "101":
            contador += 1
        if text[i:i+3] == "ABC":
            contador += 1
        if text[i:i+2] == "HO":
            contador += 1

    return contador

texto = input("Introduce una cadena: ")

print("Número de patrones:", numeroPatrones(texto))
```

Primero determinamos que el for tiene la longitud de la cadena de texto con range(len(text)), a continuación pasa todo a mayúsculas para que no haya distinciones con .upper().
El siguiente paso es el más importante en el que voy a explicar la solución de que "Un caràcter pot formar part de més d'un patró oposat" con el ejemplo de "000":
Empezamos el bucle con i=0, entramos en la primera condición donde cojemos la posición i=0 y acaba la comprobación en la posición i=0+2 y comprobaremos que efectivamente las dos primeras posiciones contienen 00.
Las siguientes condiciones no se cumplirán ya que i sigue siendo = 0 y esa primera posición contiene 0, ni 1, ni A, ni H, al igual que sus otras respectivas posiciones.
A continuación pasamos al punto más importante y comprobaremos como si que cogeria el segundo 00 dentro de la cadena 000:
En la siguiente comprobación i=1, por lo que en la primera condición cogería el segundo 0 de los tres y luego como i=1+2 es = 3 cogería también el tercer 0, cumpliendose un segundo patrón 00 dentro de los tres 0.

```
igngiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/igngiltar/Escritorio/Python/UD00-2.1.py
Introduce una cadena: 000
Número de patrones: 2
```

4.4 PREGUNTA ENTREVISTA 04

Realitza una aplicació que reba en codi mitjançant un array bidimensional (de longitud variable) un escenari de "Buscaminas", on hi haja un 0 on no hi ha mines i un -1 on si hi ha. Per a cada casella que no tinga una mina, diga quantes mines adjacents hi ha (en diagonal, horitzontal i vertical). Internament, el programa realitzarà les accions amb una funció definida com a "def contandoMinas(miCampo)" que retornarà un array bidimensional amb el nombre de mines
adjacents en cada posició.
Exemple Entrada (llegit de teclat)
0 0 -1 0
0 -1 -1 0
Exemple Eixida (mostrat per pantalla)
1 3 -1 2
1 -1 -1 2

Primero determinamos y recorremos la longitud de las filas usando un len con el array que le pasamos, y para saber la longitud de las columnas comprobaremos cuantos valores tiene la primera fila del array.
Una vez recorremos cada posición comprobaremos si hay una mina (-1), para comprobar o no las posiciones adyacentes.
Cuando no es -1 significa que es un 0, por lo que empezaremos a contar las minas de alrededor, esto va a ser gracias a los rangos determinados en el siguiente doble bucle.
Uno de los problemas que tenemos que comprobar es el estar en una esquina del mapa (osea que la fila o la columna esté en la primera o última posición), ya que parte de las posiciones que miraremos no serán válidas en nuestra matriz, por lo que lo comprobaremos con la primera condición dentro del doble bucle.
Otro problema sería contarnos a nosotros mismos, es decir a la propia casilla, lo cual comprobaremos con la segunda condición dentro del doble bucle, si se dan alguno de los dos casos no contará y pasará a la siguiente grácias al continúe.
Por último si no se dan las otras condiciones y encontramos un -1 le sumaremos 1 al contador.

```
def contandoMinas(miCampo):

    resultado = []

    for fila in range(len(miCampo)):
        nuevaFila = []

        for columna in range(len(miCampo[0])):

            if miCampo[fila][columna] == -1:
                nuevaFila.append(-1)

            else:
                contador = 0
```

```
                for i in range(fila - 1, fila + 2):
                    for j in range(columna - 1, columna + 2):

                        if i >= 0 and i < len(miCampo) and j >= 0 and j
< len(miCampo[0]):

                            if i == fila and j == columna:
                                continue

                            if miCampo[i][j] == -1:
                                contador += 1

                nuevaFila.append(contador)

        resultado.append(nuevaFila)

    return resultado



miCampo = [
    [0, 0, -1, 0],
    [0, -1, -1, 0]
]

resultado = contandoMinas(miCampo)

for fila in resultado:
    print(*fila)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/UD00_Buscaminas.py
1 3 -1 2
1 -1 -1 2
```

4.5 PREGUNTA ENTREVISTA 05

Dissenya la classe "Escola" per a un programa que permeta la gestió completa de diverses escoles.
Per cada escola, s'instanciara la classe "Escola" que definim ara:
● Classe Escola: contindrà la informació de les escoles (nom, localitat, responsable...) així com dels diferents professors i com els diferents grups d'alumnes, utilitzant les següents classes proposades.
○ Ha de tindre mètodes per afegir/eliminar alumnes, professors.
De forma auxiliar a la classe "Escola", s'han de crear les següents classes:

● Classe Professor: contindrà informació dels professors que treballen allí (nom, tipus
(ciències, lletres o mixt)).
● Classe Alumne: contindrà la informació dels alumnes de l'escola (nom, curs, professor responsable (només un professor)).
No hi ha un alumne que vaja a dues escoles diferents, ni dos professors que treballen en diferents escoles. Per tant, trieu l'estructura d'objectes més adequada per al seu correcte emmagatzematge.
Les classes han d'estar declarades en un fitxer separat del programa. A més:
● S'han de representar les relacions entre classes.
● Han de tindre mètodes per gestionar correctament tots els elements, amb les seues corresponents operacions (inserció, modificació, esborrat, visualització, etc.).
Finalment, fes un codi d'exemple que permeta provar l'has capacitats del teu disseny

Este es el archivo de clases:

```
class Professor:

    # Constructor
    def __init__(self, nom, tipus):
        self.nom = nom
        self.tipus = tipus

    # Muestra la información del profesor
    def __str__(self):
        return f"Professor: {self.nom} - Tipus: {self.tipus}"


class Alumne:

    # Constructor
    def __init__(self, nom, curs, professor):
        self.nom = nom
        self.curs = curs
        self.professor = professor

    # Muestra la información del alumno
    def __str__(self):
            return f"Alumne: {self.nom} - Curs: {self.curs} - Professor:
{self.professor.nom}"


class Escola:

    # Constructor
```

```
    def __init__(self, nom, localitat, responsable):
        self.nom = nom
        self.localitat = localitat
        self.responsable = responsable

        # Lista de profesores de la escuela
        self.professors = []

        # Lista de alumnos de la escuela
        self.alumnes = []

    # Añade un profesor a la escuela
    def afegirProfessor(self, professor):
        if professor not in self.professors:
            self.professors.append(professor)

    # Elimina un profesor
    def eliminarProfessor(self, professor):

        # Comprobamos si tiene alumnos asignados
        for alumne in self.alumnes:
            if alumne.professor == professor:
                print("No es pot eliminar el professor perquè té alumnes
assignats.")
                return

        if professor in self.professors:
            self.professors.remove(professor)

    # Añade un alumno a la escuela
    def afegirAlumne(self, alumne):

        # El profesor debe pertenecer a esta escuela
        if alumne.professor not in self.professors:
                print("No es pot afegir l'alumne perquè el professor no
pertany a aquesta escola.")
            return

        # Evitamos alumnos repetidos
        if alumne not in self.alumnes:
            self.alumnes.append(alumne)

    # Elimina un alumno
```

```
    def eliminarAlumne(self, alumne):
        if alumne in self.alumnes:
            self.alumnes.remove(alumne)

    # Modifica los datos de un alumno
    def modificarAlumne(self, alumne, nouNom, nouCurs):
        if alumne in self.alumnes:
            alumne.nom = nouNom
            alumne.curs = nouCurs

    # Muestra todos los profesores
    def mostrarProfessors(self):
        for professor in self.professors:
            print(professor)

    # Muestra todos los alumnos
    def mostrarAlumnes(self):
        for alumne in self.alumnes:
            print(alumne)

    # Muestra la información de la escuela
    def __str__(self):
            return f"Escola: {self.nom} - Localitat: {self.localitat} -
Responsable: {self.responsable}"
```

Este es el main de prueba:

```
from UD00_escola_clases import Professor, Alumne, Escola

# CREACIÓN DE VARIAS ESCUELAS
escola1 = Escola("IES Doctor Faustí Barberà", "Alaquàs", "Pilar")
escola2 = Escola("IES Salvador Gadea", "Aldaia", "Juan")
escola3 = Escola("Olivar", "Alaquàs", "Jose Luis")

# Guardamos todas las escuelas en una lista
escoles = [escola1, escola2, escola3]

# CREACIÓN DE PROFESORES
profesor1 = Professor("Oscar", "ciències")
profesor2 = Professor("Enrrique", "lletres")
profesor3 = Professor("Borja", "mixt")
```

```
profesor4 = Professor("Juan", "ciències")
profesor5 = Professor("Micaela", "lletres")

profesor6 = Professor("Ricard", "mixt")
profesor7 = Professor("Paqui", "ciències")

# ASIGNAMOS PROFESORES A CADA ESCUELA
# Escola 1
escola1.afegirProfessor(profesor1)
escola1.afegirProfessor(profesor2)
escola1.afegirProfessor(profesor3)

# Escola 2
escola2.afegirProfessor(profesor4)
escola2.afegirProfessor(profesor5)

# Escola 3
escola3.afegirProfessor(profesor6)
escola3.afegirProfessor(profesor7)

# CREACIÓN DE ALUMNOS
alumne1 = Alumne("Nacho", "2° DAM", profesor1)
alumne2 = Alumne("Angel", "2° DAM", profesor2)
alumne3 = Alumne("Jorge", "1° DAM", profesor1)

alumne4 = Alumne("Carlos", "2° DAW", profesor4)
alumne5 = Alumne("Marta", "1° DAW", profesor5)

alumne6 = Alumne("Sergio", "2° SMR", profesor6)
alumne7 = Alumne("Ivan", "1° SMR", profesor7)

# AÑADIMOS ALUMNOS A SUS ESCUELAS
# Escola 1
escola1.afegirAlumne(alumne1)
escola1.afegirAlumne(alumne2)
escola1.afegirAlumne(alumne3)

# Escola 2
escola2.afegirAlumne(alumne4)
escola2.afegirAlumne(alumne5)

# Escola 3
escola3.afegirAlumne(alumne6)
```

```
escola3.afegirAlumne(alumne7)

# MOSTRAMOS TODAS LAS ESCUELAS
print("=========================================")
print("           TOTES LES ESCOLES")
print("=========================================")

for escola in escoles:

   print("\n" + str(escola))

   print("\nProfessors:")
   escola.mostrarProfessors()

   print("\nAlumnes:")
   escola.mostrarAlumnes()

# MODIFICACIÓN DE UN ALUMNO
print("\n=========================================")
print("       MODIFICACIÓ D'UN ALUMNE")
print("=========================================")

print("Abans:")
print(alumne1)

escola1.modificarAlumne(
   alumne1,
   "Ignacio",
   "2º DAM"
)

print("Després:")
print(alumne1)

# INTENTAMOS AÑADIR UN ALUMNO CON UN PROFESOR DE OTRA ESCUELA
print("\n=========================================")
print("       PROVA DE RESTRICCIÓ")
print("=========================================")

alumneIncorrecte = Alumne(
   "Pedro",
   "2º DAM",
   profesor4
```

```
)

print("Intentant afegir Pedro a Escola 1...")

escola1.afegirAlumne(alumneIncorrecte)

# INTENTAMOS ELIMINAR UN PROFESOR QUE TIENE ALUMNOS
print("\n=========================================")
print("   INTENTAR ELIMINAR PROFESSOR AMB ALUMNES")
print("=========================================")

escola1.eliminarProfessor(profesor1)

# ELIMINAMOS UN ALUMNO
print("\n=========================================")
print("          ELIMINAR ALUMNE")
print("=========================================")

print("Alumnes abans:")
escola1.mostrarAlumnes()

escola1.eliminarAlumne(alumne3)

print("\nAlumnes després:")
escola1.mostrarAlumnes()

# AHORA PODEMOS ELIMINAR EL PROFESOR PORQUE YA NO TIENE ALUMNOS
print("\n=========================================")
print("       ELIMINAR PROFESSOR")
print("=========================================")

# Profesor 3 no tiene alumnos
escola1.eliminarProfessor(profesor3)

print("Professors de Escola 1:")
escola1.mostrarProfessors()

# INTENTAMOS AÑADIR DOS VECES EL MISMO PROFESOR
print("\n=========================================")
print("       PROVA DE DUPLICATS")
print("=========================================")

print("Professors abans:")
```

```
escola2.mostrarProfessors()

escola2.afegirProfessor(profesor4)
escola2.afegirProfessor(profesor4)

print("\nProfessors després:")
escola2.mostrarProfessors()


# MOSTRAMOS EL RESULTADO FINAL
print("\n=========================================")
print("            ESTAT FINAL")
print("=========================================")

for escola in escoles:

   print("\n" + str(escola))

   print("\nProfessors:")
   escola.mostrarProfessors()

   print("\nAlumnes:")
   escola.mostrarAlumnes()
```

Aquí podemos observar el correcto funcionamiento:

```
igngiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/igngiltar/Escritorio/Python/UD00_escola_main.py
=======================================
          TOTES LES ESCOLES
=======================================

Escola: IES Doctor Faustí Barberà - Localitat: Alaquàs - Responsable: Pilar

Professors:
Professor: Oscar - Tipus: ciències
Professor: Enrrique - Tipus: lletres
Professor: Borja - Tipus: mixt

Alumnes:
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar

Escola: IES Salvador Gadea - Localitat: Aldaia - Responsable: Juan

Professors:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

Alumnes:
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela

Escola: Olivar - Localitat: Alaquàs - Responsable: Jose Luis

Professors:
Professor: Ricard - Tipus: mixt
Professor: Paqui - Tipus: ciències

Alumnes:
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui
```

```
========================================
       MODIFICACIÓ D'UN ALUMNE
========================================
Abans:
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Després:
Alumne: Ignacio - Curs: 2º DAM - Professor: Oscar

========================================
       PROVA DE RESTRICCIÓ
========================================
Intentant afegir Pedro a Escola 1...
No es pot afegir l'alumne perquè el professor no pertany a aquesta escola.

========================================
   INTENTAR ELIMINAR PROFESSOR AMB ALUMNES
========================================
No es pot eliminar el professor perquè té alumnes assignats.

========================================
         ELIMINAR ALUMNE
========================================
Alumnes abans:
Alumne: Ignacio - Curs: 2º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar

Alumnes després:
Alumne: Ignacio - Curs: 2º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
```

```
========================================
       ELIMINAR PROFESSOR
========================================
Professors de Escola 1:
Professor: Oscar - Tipus: ciències
Professor: Enrrique - Tipus: lletres

========================================
       PROVA DE DUPLICATS
========================================
Professors abans:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

Professors després:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

========================================
           ESTAT FINAL
========================================

Escola: IES Doctor Faustí Barberà - Localitat: Alaquàs - Responsable: Pilar

Professors:
Professor: Oscar - Tipus: ciències
Professor: Enrrique - Tipus: lletres

Alumnes:
Alumne: Ignacio - Curs: 2º DAM - Professor: Oscar
```

```
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique

Escola: IES Salvador Gadea - Localitat: Aldaia - Responsable: Juan

Professors:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

Alumnes:
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela

Escola: Olivar - Localitat: Alaquàs - Responsable: Jose Luis

Professors:
Professor: Ricard - Tipus: mixt
Professor: Paqui - Tipus: ciències

Alumnes:
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui
```

4.6 PREGUNTA ENTREVISTA 06

SQLAlchemy és una biblioteca de Python amb diverses funcions i drivers per tractar amb bases de dades. Una de les funcions més destacades és la inclusió d'un ORM (Object Relational Mapping).
Partint del disseny de classes de la pregunta anterior i el seu codi, fes un nou programa que afegisca persistència a les classes, mantenint les seues relacions utilitzant l'ORM de Python 3 inclòs a la biblioteca SQLAlchemy amb el driver de SQLite. Fes un codi d'exemple que permeta provar la persistència adquirida en implementar l'ORM amb objectes de la classe "Alumne", indicant clarament en comentaris que ha de fer-se per provar-la.

El primer paso será instalar SQLAlchemy:

```
PS C:\Users\Nacho\AppData\Local\Programs\Microsoft VS Code> pip3 install SQLAlchemy
Collecting SQLAlchemy
  Downloading sqlalchemy-2.1.3-cp314-cp314-win_amd64.whl.metadata (9.9 kB)
Collecting typing-extensions>=4.6.0 (from SQLAlchemy)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Downloading sqlalchemy-2.1.3-cp314-cp314-win_amd64.whl (2.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.4/2.4 MB 7.1 MB/s  0:00:00
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Installing collected packages: typing-extensions, SQLAlchemy
Successfully installed SQLAlchemy-2.1.3 typing-extensions-4.16.0
```

Este será nuestro main:

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

from UD00_escola_clases import Base, Professor, Alumne, Escola

# CONEXIÓN CON LA BASE DE DATOS
# Creamos la conexión con SQLite.
# Si el archivo escola.db no existe, se creará automáticamente.
engine = create_engine("sqlite:///escola.db")

# Creamos las tablas de la base de datos
Base.metadata.create_all(engine)

# Creamos una sesión para trabajar con la base de datos
session = Session(engine)

# CREACIÓN DE VARIAS ESCUELAS
escola1 = Escola(
    "IES Doctor Faustí Barberà",
    "Alaquàs",
    "Pilar"
)
```

```
escola2 = Escola(
    "IES Salvador Gadea",
    "Aldaia",
    "Juan"
)

escola3 = Escola(
    "Olivar",
    "Alaquàs",
    "Jose Luis"
)

# CREACIÓN DE PROFESORES
profesor1 = Professor("Oscar", "ciències")
profesor2 = Professor("Enrrique", "lletres")
profesor3 = Professor("Borja", "mixt")

profesor4 = Professor("Juan", "ciències")
profesor5 = Professor("Micaela", "lletres")

profesor6 = Professor("Ricard", "mixt")
profesor7 = Professor("Paqui", "ciències")


# ASIGNAMOS PROFESORES A CADA ESCUELA

# Escola 1
escola1.afegirProfessor(profesor1)
escola1.afegirProfessor(profesor2)
escola1.afegirProfessor(profesor3)

# Escola 2
escola2.afegirProfessor(profesor4)
escola2.afegirProfessor(profesor5)

# Escola 3
escola3.afegirProfessor(profesor6)
escola3.afegirProfessor(profesor7)

# CREACIÓN DE ALUMNOS
alumne1 = Alumne("Nacho", "2º DAM", profesor1)
alumne2 = Alumne("Angel", "2º DAM", profesor2)
```

```
alumne3 = Alumne("Jorge", "1º DAM", profesor1)

alumne4 = Alumne("Carlos", "2º DAW", profesor4)
alumne5 = Alumne("Marta", "1º DAW", profesor5)

alumne6 = Alumne("Sergio", "2º SMR", profesor6)
alumne7 = Alumne("Ivan", "1º SMR", profesor7)

# AÑADIMOS ALUMNOS A SUS ESCUELAS

# Escola 1
escola1.afegirAlumne(alumne1)
escola1.afegirAlumne(alumne2)
escola1.afegirAlumne(alumne3)

# Escola 2
escola2.afegirAlumne(alumne4)
escola2.afegirAlumne(alumne5)

# Escola 3
escola3.afegirAlumne(alumne6)
escola3.afegirAlumne(alumne7)


# GUARDAMOS TODOS LOS OBJETOS EN LA BASE DE DATOS
session.add_all([
    escola1,
    escola2,
    escola3
])

session.commit()


# MOSTRAMOS LOS DATOS GUARDADOS
print("\n=========================================")
print("           DADES DE LA BASE DE DADES")
print("=========================================")


# Recuperamos las escuelas desde la base de datos
escoles = session.query(Escola).all()
```

```
for escola in escoles:

    print("\n" + str(escola))

    print("\nProfessors:")
    escola.mostrarProfessors()

    print("\nAlumnes:")
    escola.mostrarAlumnes()


print("\n=========================================")
print("       ALUMNES RECUPERATS DE LA BD")
print("=========================================")


# Recuperamos directamente los alumnos de SQLite
alumnes = session.query(Alumne).all()


for alumne in alumnes:
    print(alumne)


# Cerramos la sesión
session.close()
```

Ejecutamos la primera prueba:

```
PS C:\Users\Nacho\Desktop\SGE> py UD00_escola_main.py
========================================
          DADES DE LA BASE DE DADES
========================================

Escola: IES Doctor Faustí Barberà - Localitat: Alaquàs - Responsable: Pilar

Professors:
Professor: Oscar - Tipus: ciències
Professor: Enrrique - Tipus: lletres
Professor: Borja - Tipus: mixt

Alumnes:
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
```

```
Escola: IES Salvador Gadea - Localitat: Aldaia - Responsable: Juan

Professors:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

Alumnes:
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela

Escola: Olivar - Localitat: Alaquàs - Responsable: Jose Luis

Professors:
Professor: Ricard - Tipus: mixt
Professor: Paqui - Tipus: ciències

Alumnes:
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui

======================================
      ALUMNES RECUPERATS DE LA BD
======================================
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui
```

Aquí podemos comprobar como se ha creado la base de datos escola.db

```
PS C:\Users\Nacho\Desktop\SGE> ls


    Directorio: C:\Users\Nacho\Desktop\SGE


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        04/10/2026     18:45                __pycache__
-a----        04/10/2026     18:45          16384 escola.db
-a----        04/10/2026     18:40           5094 UD00_escola_clases.py
-a----        04/10/2026     18:43           3074 UD00_escola_main.py
```

A continuación comprobaremos la persistencia, para ello lo primero que tenemos que hacer es comentar la siguiente parte del código donde creamos los datos:

```
'''
# CREACIÓN DE VARIAS ESCUELAS
escola1 = Escola(
    "IES Doctor Faustí Barberà",
    "Alaquàs",
    "Pilar"
)
escola2 = Escola(
```

```
    "IES Salvador Gadea",
    "Aldaia",
    "Juan"
)

escola3 = Escola(
    "Olivar",
    "Alaquàs",
    "Jose Luis"
)

# CREACIÓN DE PROFESORES
profesor1 = Professor("Oscar", "ciències")
profesor2 = Professor("Enrrique", "lletres")
profesor3 = Professor("Borja", "mixt")

profesor4 = Professor("Juan", "ciències")
profesor5 = Professor("Micaela", "lletres")

profesor6 = Professor("Ricard", "mixt")
profesor7 = Professor("Paqui", "ciències")


# ASIGNAMOS PROFESORES A CADA ESCUELA
# Escola 1
escola1.afegirProfessor(profesor1)
escola1.afegirProfessor(profesor2)
escola1.afegirProfessor(profesor3)

# Escola 2
escola2.afegirProfessor(profesor4)
escola2.afegirProfessor(profesor5)

# Escola 3
escola3.afegirProfessor(profesor6)
escola3.afegirProfessor(profesor7)

# CREACIÓN DE ALUMNOS
alumne1 = Alumne("Nacho", "2° DAM", profesor1)
alumne2 = Alumne("Angel", "2° DAM", profesor2)
alumne3 = Alumne("Jorge", "1° DAM", profesor1)

alumne4 = Alumne("Carlos", "2° DAW", profesor4)
```

```
alumne5 = Alumne("Marta", "1° DAW", profesor5)

alumne6 = Alumne("Sergio", "2° SMR", profesor6)
alumne7 = Alumne("Ivan", "1° SMR", profesor7)

# AÑADIMOS ALUMNOS A SUS ESCUELAS

# Escola 1
escola1.afegirAlumne(alumne1)
escola1.afegirAlumne(alumne2)
escola1.afegirAlumne(alumne3)

# Escola 2
escola2.afegirAlumne(alumne4)
escola2.afegirAlumne(alumne5)

# Escola 3
escola3.afegirAlumne(alumne6)
escola3.afegirAlumne(alumne7)

# GUARDAMOS TODOS LOS OBJETOS EN LA BASE DE DATOS
session.add_all([
    escola1,
    escola2,
    escola3
])

session.commit()
'''
```

Al ejecutar el programa podemos comprobar cómo siguen apareciendo los datos:

```
PS C:\Users\Nacho\Desktop\SGE> py UD00_escola_main.py
========================================
          DADES DE LA BASE DE DADES
========================================

Escola: IES Doctor Faustí Barberà - Localitat: Alaquàs - Responsable: Pilar

Professors:
Professor: Oscar - Tipus: ciències
Professor: Enrrique - Tipus: lletres
Professor: Borja - Tipus: mixt

Alumnes:
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
```

```
Escola: IES Salvador Gadea - Localitat: Aldaia - Responsable: Juan

Professors:
Professor: Juan - Tipus: ciències
Professor: Micaela - Tipus: lletres

Alumnes:
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela

Escola: Olivar - Localitat: Alaquàs - Responsable: Jose Luis

Professors:
Professor: Ricard - Tipus: mixt
Professor: Paqui - Tipus: ciències

Alumnes:
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui

========================================
      ALUMNES RECUPERATS DE LA BD
========================================
Alumne: Nacho - Curs: 2º DAM - Professor: Oscar
Alumne: Jorge - Curs: 1º DAM - Professor: Oscar
Alumne: Angel - Curs: 2º DAM - Professor: Enrrique
Alumne: Carlos - Curs: 2º DAW - Professor: Juan
Alumne: Marta - Curs: 1º DAW - Professor: Micaela
Alumne: Sergio - Curs: 2º SMR - Professor: Ricard
Alumne: Ivan - Curs: 1º SMR - Professor: Paqui
```

Lo siguiente será comprobar que una modificación se guarda, para ello añadimos la modificación de un alumno antes del session.close (es importante dejar comentado el código donde creamos los datos)

```python
# PRUEBA DE MODIFICACIÓN DE UN ALUMNE

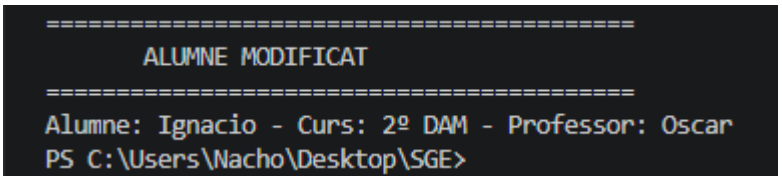
# Buscamos el alumno Nacho en la base de datos
alumne = session.query(Alumne).filter_by(nom="Nacho").first()

# Modificamos su nombre
if alumne:
    alumne.nom = "Ignacio"
    session.commit()

    print("\n========================================")
    print("       ALUMNE MODIFICAT")
    print("========================================")
    print(alumne)

# Cerramos la sesión
session.close()
```

Al ejecutarlo podemos ver que al final del programa nos aparece la modificación:



Por último vamos a comprobar que se guarda la modificación, para ello vamos a comentar la última parte del código que hemos añadido para modificar el alumno y volveremos a ejecutar el programa

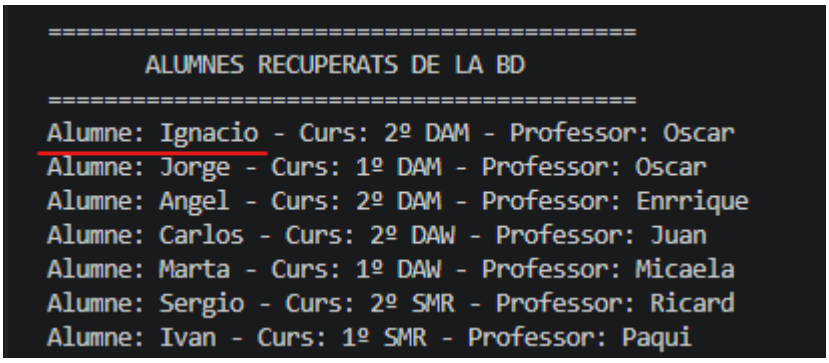
```
'''
# PRUEBA DE MODIFICACIÓN DE UN ALUMNE

# Buscamos el alumno Nacho en la base de datos
alumne = session.query(Alumne).filter_by(nom="Nacho").first()

# Modificamos su nombre
if alumne:
    alumne.nom = "Ignacio"
    session.commit()

    print("\n=========================================")
    print("       ALUMNE MODIFICAT")
    print("=========================================")
    print(alumne)
'''
```

Como podemos comprobar al ejecutar el programa sí que aparece el alumno modificado:



4.7 PREGUNTA ENTREVISTA 07

Realitza un programa que rebrà per paràmetres de consola dos noms: el del fitxer d'entrada i el del fitxer d'eixida.
A més, per tal de tindre aquest programa en un executable i poder distribuir-ho haurem d'empacar el programa en un executable utilitzant la biblioteca https://www.pyinstaller.org/.
El programa implementarà dues funcions: "esPalindromo" i "esPrimo".
Important: aquestes funcions hauran d'estar comprovades amb un test unitari. Recomanem peraquest cas "doctest" https://docs.python.org/es/3/library/doctest.html però val qualsevol altra
utilitat per fer tests unitaris. El programa ha de llegir el fitxer d'entrada. Aquest fitxer conte un número per línia (no s'especifica quantes línies, ha de funcionar per qualsevol número). El nostre programa ha d'imprimir en el fitxer d'eixida dues línies i després un llistat amb un número
per línia:
● La primera línia ha de dir quants números eren "palíndroms" (capicua)
● Una altra línia dient quants eren "cosins".
● Finalment, "n" línies indicant quins "n" números eren al mateix temps cosins i palíndroms.
Per exemple:
Fitxer "exempleEntrada.txt"
5
7
11
13
14
15
22
Fitxer "exempleEixida.txt"
Hi han 4 números palíndroms.
Hi han 4 números cosins.
5
7
11

```
import sys
import doctest

# Comprueba si un número es palíndromo
def esPalindromo(numero):
    """
    Comprueba si un número es palíndromo.

    >>> esPalindromo(5)
    True
    >>> esPalindromo(11)
```

```
    True
    >>> esPalindromo(15)
    False
    >>> esPalindromo(121)
    True
    """

    # Convertimos el número a texto para poder invertirlo
    numero = str(numero)

    # Comparamos el número original con el número invertido
    return numero == numero[::-1]


# Comprueba si un número es primo
def esPrimo(numero):
    """
    Comprueba si un número es primo.

    >>> esPrimo(2)
    True
    >>> esPrimo(5)
    True
    >>> esPrimo(7)
    True
    >>> esPrimo(10)
    False
    >>> esPrimo(1)
    False
    """

    # Los números menores que 2 no son primos
    if numero < 2:
        return False

    # Comprobamos si tiene algún divisor
    for i in range(2, numero):
        if numero % i == 0:
            return False

    return True
```

```
# Ejecuta las pruebas escritas en las funciones
doctest.testmod()


# Comprobamos que se han recibido los dos archivos
if len(sys.argv) != 3:
   print("Ús: python3 UD00_palindromo.py fitxerEntrada fitxerEixida")
   sys.exit()


# Obtenemos los nombres de los archivos desde la consola
archivoEntrada = sys.argv[1]
archivoSalida = sys.argv[2]


# Guardamos los números del archivo en una lista
numeros = []

with open(archivoEntrada, "r") as archivo:

   # Leemos el archivo línea por línea
   for linea in archivo:
       numero = int(linea.strip())
       numeros.append(numero)


# Variables para guardar los resultados
cantidadPalindromos = 0
cantidadPrimos = 0
numerosPrimosPalindromos = []


# Comprobamos cada número
for numero in numeros:

   # Contamos los números palíndromos
   if esPalindromo(numero):
       cantidadPalindromos += 1

   # Contamos los números primos
   if esPrimo(numero):
       cantidadPrimos += 1
```

```
    # Guardamos los que cumplen las dos condiciones
    if esPalindromo(numero) and esPrimo(numero):
        numerosPrimosPalindromos.append(numero)


# Escribimos los resultados en el archivo de salida
with open(archivoSalida, "w") as archivo:

    archivo.write(
        f"Hi han {cantidadPalindromos} números palíndromos.\n"
    )

    archivo.write(
        f"Hi han {cantidadPrimos} números primos.\n"
    )

    # Escribimos un número por línea
    for numero in numerosPrimosPalindromos:
        archivo.write(f"{numero}\n")
```

Podemos observar que después de ejecutar el programa se crea el fichero de salida correctamente:

```
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.7_palindromo$ ls
ejemploEntrada.txt  UD00_palindromo.py
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.7_palindromo$ python3 UD00_palindromo.py ejemploEntrada.txt ejemploEixida.txt
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.7_palindromo$ ls
ejemploEixida.txt  ejemploEntrada.txt  UD00_palindromo.py
```

Este es el fichero de entrada

UD00_4.7_palindromo > ejemploEntrada.txt

```
1  5
2  7
3  11
4  13
5  14
6  15
7  22
```

Aquí podemos ver el fichero de salida generado

UD00_4.7_palindromo > ejemploEixida.txt

```
1  Hi han 4 números palíndromos.
2  Hi han 4 números primos.
3  5
4  7
5  11
```

4.8 PREGUNTA ENTREVISTA 08

Crea un programa Python 3 que ens ajude a organitzar els fitxers de la carpeta actual. El programa tindrà una llista amb extensions de programes (ha de funcionar amb qualsevol).
El programa en executar-se, per cada extensió, mourà tots els fitxers amb eixa extensió a una carpeta amb eixe nom.
Per exemple, amb llista=[“png”,”mp4”, “doc”], el programa mouria tots els fitxers amb extensió “.png” de la carpeta actual a una carpeta anomenada “png”. El mateix amb els fitxers “.mp4” a la carpeta “mp4” i els fitxers “.doc” amb la carpeta “doc”.
A més, per tal de tindre aquest programa en un executable i poder distribuir-ho haurem d’empacar el programa en un executable utilitzant la biblioteca https://www.pyinstaller.org/

```
import os
import shutil

# Lista de extensiones que queremos organizar
lista = ["png", "mp4", "doc"]

# Obtenemos los archivos de la carpeta actual
archivos = os.listdir(".")

# Recorremos cada extensión de la lista
for extension in lista:

   # Creamos el nombre de la carpeta
   carpeta = extension

   # Si la carpeta no existe, la creamos
   if not os.path.exists(carpeta):
       os.mkdir(carpeta)

   # Recorremos todos los elementos de la carpeta actual
   for archivo in archivos:

       # Comprobamos que sea un archivo
       if os.path.isfile(archivo):

           # Obtenemos la extensión del archivo
           extensionArchivo = os.path.splitext(archivo)[1]

           # Quitamos el punto de la extensión
           extensionArchivo = extensionArchivo.replace(".", "")
```

```
                # Comparamos la extensión del archivo con la extensión
actual
            if extensionArchivo.lower() == extension.lower():

                # Creamos la ruta de destino
                destino = os.path.join(carpeta, archivo)

                # Movemos el archivo
                shutil.move(archivo, destino)

                # Mostramos qué archivo hemos movido
                print(f"Movido: {archivo} -> {carpeta}")
```

Como podemos comprobar al ejecutar el programa se crean las carpetas de cada extensión y se introducen sus respectivos ficheros dentro de ellas:

```
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ ls -l
total 24
-rw-r--r-- 1 ignguiltar Domain Users 9270 oct  2 17:58 fasuti.png
-rw-r--r-- 1 ignguiltar Domain Users    0 oct  2 17:59 prueba_nacho_4.8.doc
-rw-r--r-- 1 ignguiltar Domain Users 1241 oct  2 17:57 UD00_4.8.py
-rw-r--r-- 1 ignguiltar Domain Users 6195 oct  2 18:01 video_prueba.mp4
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/UD00_4.8/UD00_4.8.py
Movido: fasuti.png -> png
Movido: video_prueba.mp4 -> mp4
Movido: prueba_nacho_4.8.doc -> doc
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ ls -l
total 16
drwxr-xr-x 2 ignguiltar Domain Users 4096 oct  2 18:05 doc
drwxr-xr-x 2 ignguiltar Domain Users 4096 oct  2 18:05 mp4
drwxr-xr-x 2 ignguiltar Domain Users 4096 oct  2 18:05 png
-rw-r--r-- 1 ignguiltar Domain Users 1241 oct  2 17:57 UD00_4.8.py
```

```
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ cd doc
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8/doc$ ls
prueba_nacho_4.8.doc
```

```
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ cd mp4/
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8/mp4$ ls
video_prueba.mp4
```

```
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8$ cd png
ignguiltar@ORJ1:~/Escritorio/Python/UD00_4.8/png$ ls
fasuti.png
```

4.9 PREGUNTA ENTREVISTA 09

Utilitzant la biblioteca https://pypi.org/project/python-barcode/, hem de llegir un fitxer ".csv" passat per paràmetres de consola. Aquest fitxer tindrà un format en el qual cada línia usarà "nomalumne,ID".
Per exemple
"Alumne 01","1"
"Alumne 02","2"

Haurem de fer un programa, que llegint eixe fitxer “.csv” genere per cada alumne genere un fitxer “nomalumne.png” amb un codi de barres en format “EAN 13” que tinga com informació el “ID” de l’alumne.

Primero realizamos las instalaciones necesarias:

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py -m pip install python-barcode
Collecting python-barcode
  Downloading python_barcode-0.16.1-py3-none-any.whl.metadata (2.7 kB)
Downloading python_barcode-0.16.1-py3-none-any.whl (228 kB)
Installing collected packages: python-barcode
Successfully installed python-barcode-0.16.1
```

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py -m pip install pillow
Collecting pillow
  Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl (7.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.2/7.2 MB 39.3 MB/s  0:00:00
Installing collected packages: pillow
Successfully installed pillow-12.3.0
```

Creamos el CSV:

	A	B	C
1	Alumne 01	1	
2	Alumne 02	2	
3	Alumne 03	3	

```python
import sys
import csv
import barcode
from barcode.writer import ImageWriter

# Comprobamos que se haya indicado el archivo CSV
if len(sys.argv) != 2:
    print("Uso: py UD00_4.9.py prueba_4.9.csv")
    sys.exit()


# Guardamos el nombre del archivo recibido por consola
archivo = sys.argv[1]


# Abrimos el archivo CSV
with open(archivo, "r", encoding="utf-8") as fichero:

    # Leemos el CSV
    lector = csv.reader(fichero)
```

```
    # Recorremos cada alumno
    for fila in lector:

        # Obtenemos el nombre y el ID
        nombre = fila[0]
        id_alumno = fila[1]

          # EAN-13 necesita 12 dígitos para calcular automáticamente el
dígito de control, este dígito de control será el último a continuación
del ID.
        codigo = id_alumno.zfill(12)

        # Creamos el código de barras EAN-13
        ean = barcode.get(
            "ean13",
            codigo,
            writer=ImageWriter()
        )

        # Generamos el archivo PNG con el nombre del alumno
        ean.save(nombre)

        print(f"Generado: {nombre}.png")
```

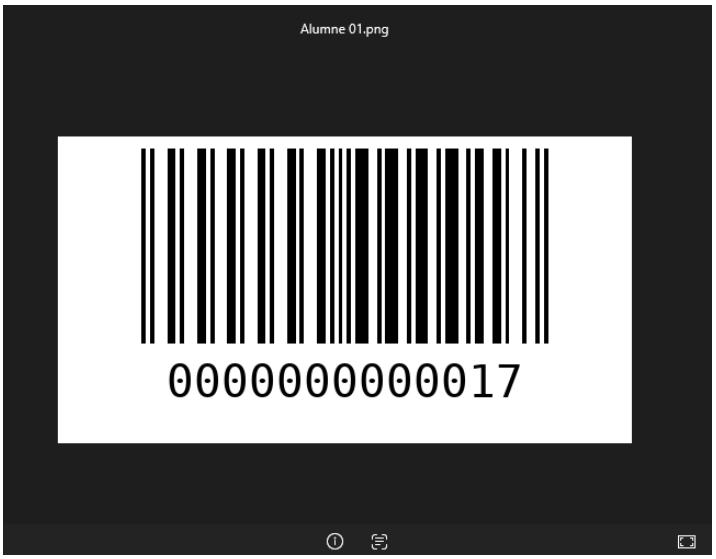
Lo ejecutamos y podemos comprobar cómo se generan los png con los códigos de barras en formato “EAN 13”

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py UD00_4.9.py prueba_4.9.csv
Generado: Alumne 01.png
Generado: Alumne 02.png
Generado: Alumne 03.png
```

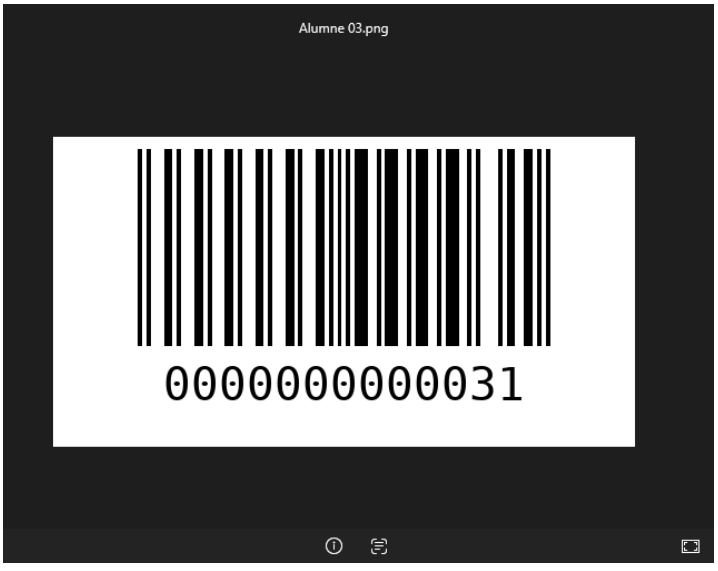
```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> ls


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        04/10/2026     20:07           2667 Alumne 01.png
-a----        04/10/2026     20:07           3063 Alumne 02.png
-a----        04/10/2026     20:07           2952 Alumne 03.png
-a----        04/10/2026     19:58             39 prueba_4.9.csv
-a----        04/10/2026     20:07           1042 UD00_4.9.py
```

Podemos ver cómo está la secuencia de 0 junto al ID y por último el dígito de control :







4.10 PREGUNTA ENTREVISTA 10

Complementant la pregunta anterior i utilitzant com a base el següent exemple de codi https://www.geeksforgeeks.org/how-to-make-a-barcode-reader-in-python/ fes un programa que llegisca tots els ".png" d'un directori i ens diga el nom d'alumne i el seu ID. (Recorda, el nom de l'alumne és el nom del ".png" i l'ID has d'extraure-lo del codi de barres)

Hacemos las instalaciones necesarias:

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py -m pip install pyzbar
Collecting pyzbar
  Downloading pyzbar-0.1.9-py2.py3-none-win_amd64.whl.metadata (10 kB)
Downloading pyzbar-0.1.9-py2.py3-none-win_amd64.whl (817 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 817.4/817.4 kB 17.3 MB/s  0:00:00
Installing collected packages: pyzbar
Successfully installed pyzbar-0.1.9
```

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py -m pip install opencv-python
Collecting opencv-python
  Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl.metadata (20 kB)
Collecting numpy>=2 (from opencv-python)
  Downloading numpy-2.5.3-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Downloading opencv_python-5.0.0.93-cp37-abi3-win_amd64.whl (44.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.0/44.0 MB 67.4 MB/s  0:00:00
Downloading numpy-2.5.3-cp314-cp314-win_amd64.whl (12.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.7/12.7 MB 30.6 MB/s  0:00:00
Installing collected packages: numpy, opencv-python
Successfully installed numpy-2.5.3 opencv-python-5.0.0.93
```

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py -m pip install opencv-contrib-python
Collecting opencv-contrib-python
  Downloading opencv_contrib_python-5.0.0.93-cp37-abi3-win_amd64.whl.metadata (20
kB)
Requirement already satisfied: numpy>=2 in C:\Users\Nacho\AppData\Local\Programs\P
ython\Python314\Lib\site-packages (from opencv-contrib-python) (2.5.3)
Downloading opencv_contrib_python-5.0.0.93-cp37-abi3-win_amd64.whl (53.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 53.8/53.8 MB 43.1 MB/s  0:00:01
Installing collected packages: opencv-contrib-python
Successfully installed opencv-contrib-python-5.0.0.93
```

```python
import os
import cv2
from pyzbar.pyzbar import decode


# Recorremos todos los archivos de la carpeta actual
for archivo in os.listdir():

    # Comprobamos que el archivo sea un PNG
```

```
    if archivo.endswith(".png"):

        # Obtenemos el nombre del alumno quitando la extensión .png
        nombre_alumno = os.path.splitext(archivo)[0]

        # Leemos la imagen
        imagen = cv2.imread(archivo)

        # Buscamos los códigos de barras de la imagen
        codigos = decode(imagen)

        # Comprobamos si se ha encontrado algún código de barras
        if codigos:

            # Recorremos los códigos encontrados
            for codigo in codigos:

                # Obtenemos la información del código de barras
                datos = codigo.data.decode("utf-8")

                # El EAN-13 tiene 13 dígitos.
                # El último es el dígito de control.
                # Los 12 primeros contienen el ID con ceros delante.
                datos_id = datos[:-1]

                   # Convertimos el ID a número para eliminar los ceros
iniciales
                id_alumno = int(datos_id)

                # Mostramos el nombre del alumno y su ID
                print(f"{nombre_alumno} -> ID: {id_alumno}")

        else:

            # Si no se encuentra ningún código de barras
            print(
                f"{nombre_alumno} -> "
                "No se ha encontrado ningún código de barras"
            )
```

Como podemos comprobar el programa está en la carpeta junto a los png generados en el ejercicio anterior y saca los ID correctamente:

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> ls

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         04/10/2026     20:07           2667 Alumne 01.png
-a----         04/10/2026     20:07           3063 Alumne 02.png
-a----         04/10/2026     20:07           2952 Alumne 03.png
-a----         04/10/2026     19:58             39 prueba_4.9.csv
-a----         04/10/2026     20:47           1496 UD00_4.10.py
-a----         04/10/2026     20:07           1042 UD00_4.9.py
```

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.9> py UD00_4.10.py
Alumne 01 -> ID: 1
Alumne 02 -> ID: 2
Alumne 03 -> ID: 3
```

4.11 PREGUNTA ENTREVISTA 11

Crea un programa que tinga una interfície gràfica" senzilla fent servir "Kivy", un entorn multiplataforma per Python https://kivy.org.
Aquest programa, en arrancar, carregara un fitxer ("users.csv") que per cada línia tindrà un usuari i el SHA1 del password, amb el següent format:
"user":"passwordinSHA1"
La interfície gràfica que tindrà només dos camps de text, "usuari" i "contrasenya" (en text pla, però oculta en teclejar) i un botó "comprovar". Si l'usuari existeix i la contrasenya és correcta, mostrarà un missatge de "OK". En cas contrari, un missatge "ERROR".

```python
import csv
import hashlib

from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.label import Label
from kivy.uix.textinput import TextInput
from kivy.uix.button import Button


class Login(App):

   def build(self):

       # Diccionario donde guardaremos los usuarios y sus SHA1
```

```
        self.usuarios = {}

        # Abrimos el archivo users.csv
        with open("users.csv", "r") as archivo:
            lector = csv.reader(archivo)

            for fila in lector:
                usuario = fila[0]
                password_sha1 = fila[1]

                self.usuarios[usuario] = password_sha1

        # Creamos el diseño de la interfaz
        layout = BoxLayout(
            orientation="vertical",
            padding=20,
            spacing=10
        )

        # Campo usuario
        self.usuario = TextInput(
            hint_text="Usuario",
            multiline=False
        )

        # Campo contraseña
        self.password = TextInput(
            hint_text="Contraseña",
            password=True,
            multiline=False
        )

        # Botón
        boton = Button(
            text="COMPROBAR"
        )

        # Etiqueta para mostrar OK o ERROR
        self.mensaje = Label(
            text=""
        )

        # Cuando pulsamos el botón llamamos a comprobar()
```

```
        boton.bind(on_press=self.comprobar)

        # Añadimos los elementos a la interfaz
        layout.add_widget(self.usuario)
        layout.add_widget(self.password)
        layout.add_widget(boton)
        layout.add_widget(self.mensaje)

        return layout

    def comprobar(self, instancia):

        usuario = self.usuario.text
        password = self.password.text

        # Comprobamos si el usuario existe
        if usuario in self.usuarios:

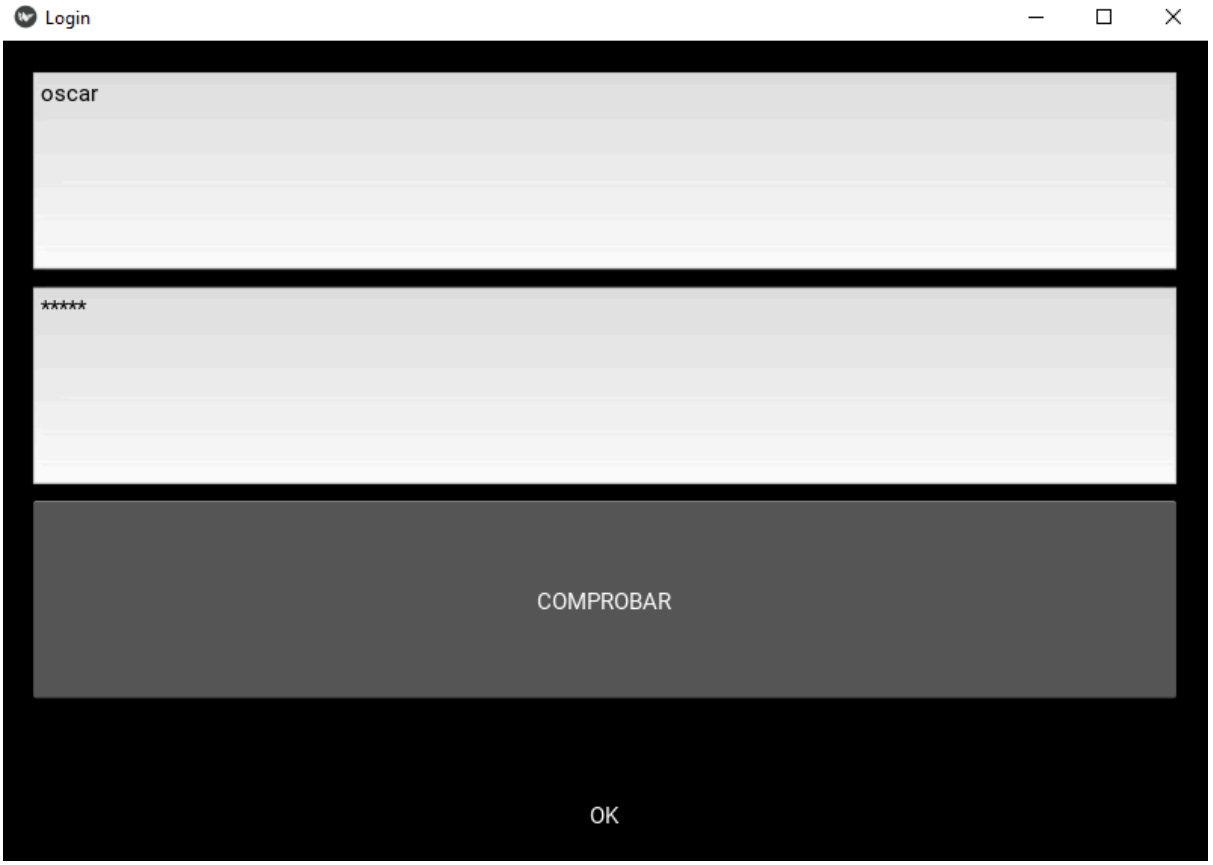
            # Convertimos la contraseña introducida a SHA1
            password_sha1 = hashlib.sha1(
                password.encode()
            ).hexdigest()

            # Comparamos el SHA1 introducido con el almacenado
            if password_sha1 == self.usuarios[usuario]:
                self.mensaje.text = "OK"
            else:
                self.mensaje.text = "ERROR"

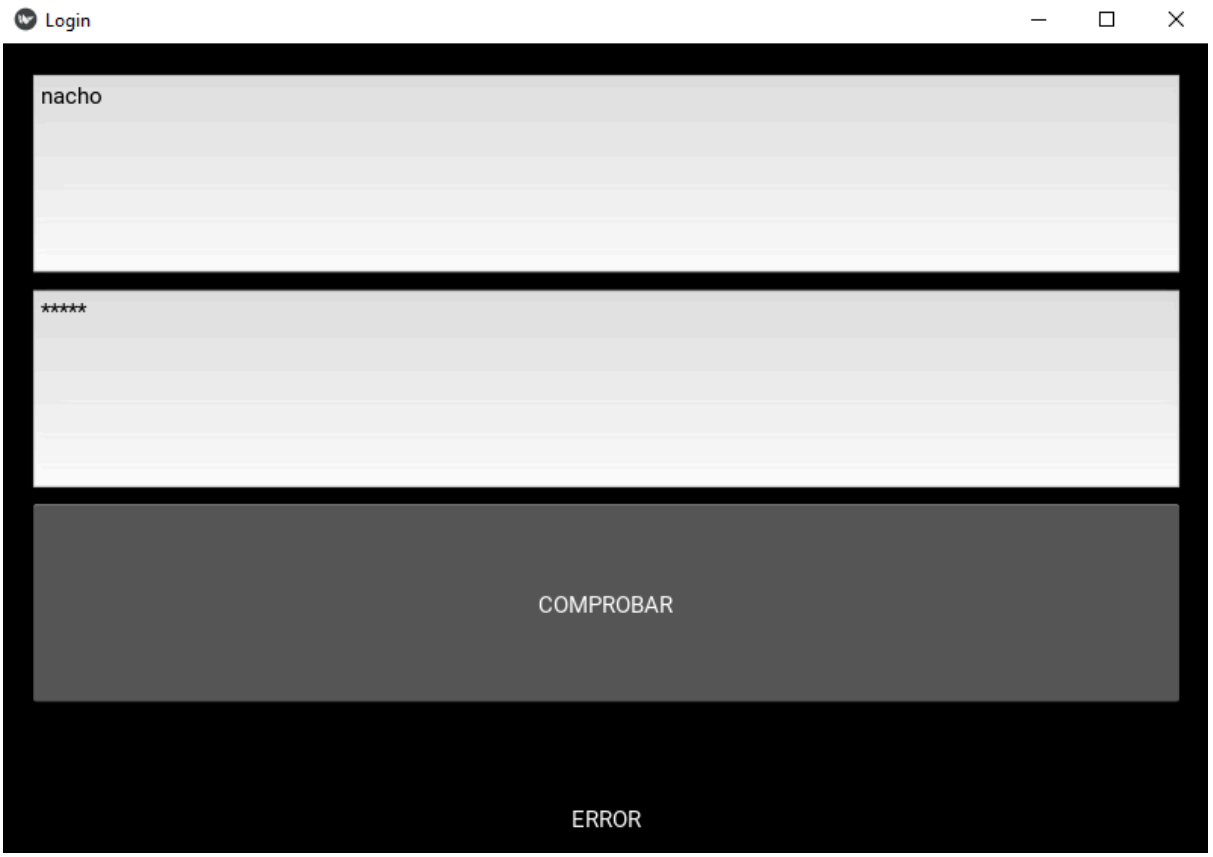
        else:
            self.mensaje.text = "ERROR"


Login().run()
```

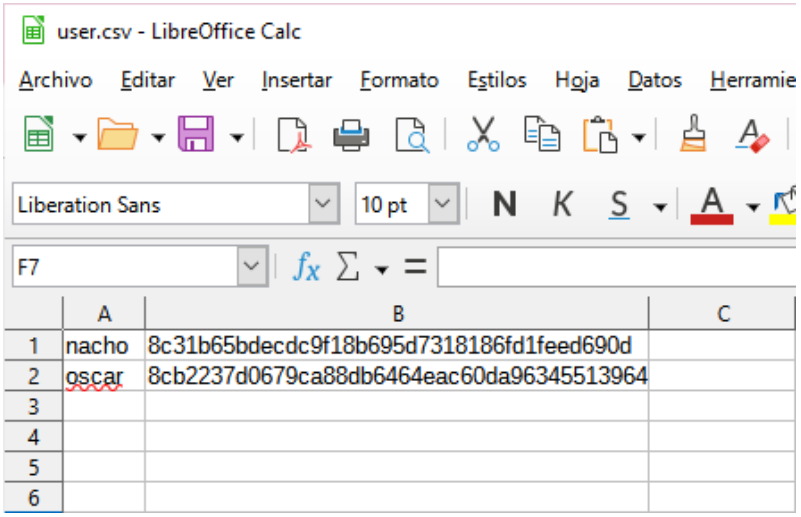
Aquí podemos ver un inicio de sesión correcto:



Y aquí uno incorrecto;



Este sería el CSV:



4.12 PREGUNTA ENTREVISTA 12

Replica aquest exemple de Python 3 per fer reconeixement de text en imatges. Afegeix comentaris per explicar el seu ús. https://towardsdatascience.com/building-a-simple-text-recognizer-in-python-93e453ddb759 Podeu provar amb alguna imatge senzilla (només text i un fons homogeni) com:https://www.facebook.com/MumukiOrg/photos/pcb.4322522981176358/4322522757843047/

Primero hacemos las instalaciones necesarias:

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.12> py -m pip install Pillow pytesseract
Requirement already satisfied: Pillow in C:\Users\Nacho\AppData\Local\Programs\Python\Python314\Lib\site-packages (12.3.0)
Collecting pytesseract
  Downloading pytesseract-0.3.13-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: packaging>=21.3 in C:\Users\Nacho\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pytesseract) (26.3)
Downloading pytesseract-0.3.13-py3-none-any.whl (14 kB)
Installing collected packages: pytesseract
Successfully installed pytesseract-0.3.13
```

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.12> winget install UB-Mannheim.TesseractOCR
El origen 'msstore' requiere que vea los siguientes contratos antes de usarlo.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
El origen requiere que la región geográfica de dos letras de la máquina actual se envíe al servicio back-end para que funcione correctamente (por ejemplo, "EE. UU.").

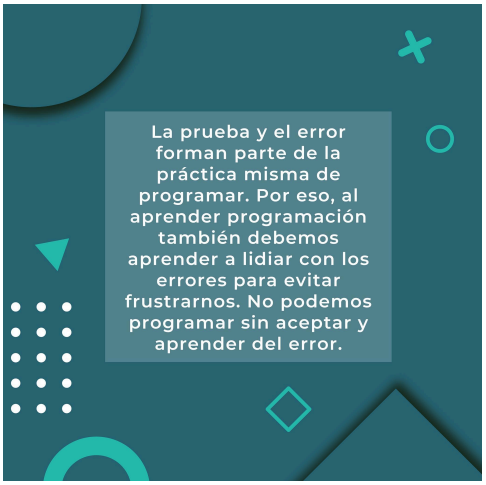
¿Está de acuerdo con todos los términos de los contratos de origen?
[Y] Sí  [N] No: Y
Encontrado Tesseract-OCR - open source OCR engine [UB-Mannheim.TesseractOCR] Versión 5.4.0.20240606
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://github.com/UB-Mannheim/tesseract/releases/download/v5.4.0.20240606/tesseract-ocr-w64-setup-5.4.0.20240606.exe
  47.8 MB / 47.8 MB
El hash del instalador se verificó correctamente
Iniciando instalación de paquete...
Instalado correctamente
```

```
UD00_4.12 > UD00_4.12.py > ...
1   from PIL import Image
2   import pytesseract
3
4   # Indicamos la ubicación de Tesseract en Windows.
5   pytesseract.pytesseract.tesseract_cmd = r"C:\Program Files\Tesseract-OCR\tesseract.exe"
6
7   # Abrimos la imagen.
8   imagen = Image.open("prueba.png")
9
10  # Reconocemos el texto de la imagen.
11  texto = pytesseract.image_to_string(imagen)
12
13  # Mostramos el texto reconocido.
14  print("Texto encontrado:")
15  print(texto)
```

Como podemos observar tenemos en la carpeta la imagen indicada de la cual obtenemos el texto:

```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.12> ls

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         04/10/2026     21:14         185644 prueba.png
-a----         04/10/2026     21:23            406 UD00_4.12.py
```




```
PS C:\Users\Nacho\Desktop\SGE\UD00_4.12> py UD00_4.12.py
Texto encontrado:
La prueba y el error
forman parte de la
practica misma de
programar. Por eso, al
aprender programacion
también debemos
aprender a lidiar con los
errores para evitar
frustrarnos. No podemos
programar sin aceptar y
aprender del error.
```

4.13 PREGUNTA ENTREVISTA 13

Utilitzant la biblioteca de Python 3 “requests” i basant-se en exemples com https://pythonizando.com/consumir-una-api-rest-con-python-3-usando-requests/ Fes un programa que consumeix l’API de “Rick and Morty” https://rickandmortyapi.com/ El nostre programa ens permetrà, demanant per teclat una espècie (atribut “specie”), obtenir un llistat de quins personatges són d’eixa espècie.

Realizamos la instalación necesaria:

```
PS C:\Users\Nacho\Desktop\SGE> py -m pip install requests
Collecting requests
  Downloading requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.5.2-cp314-cp314-win_amd64.whl.metadata (47 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Collecting urllib3<3,>=1.26 (from requests)
  Downloading urllib3-2.8.0-py3-none-any.whl.metadata (7.4 kB)
Collecting certifi>=2023.5.7 (from requests)
  Downloading certifi-2026.7.22-py3-none-any.whl.metadata (2.5 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.2-cp314-cp314-win_amd64.whl (210 kB)
Downloading idna-3.20-py3-none-any.whl (69 kB)
Downloading urllib3-2.8.0-py3-none-any.whl (135 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Installing collected packages: urllib3, idna, charset_normalizer, certifi, request
s
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.2 idna-3.20 reques
ts-2.34.2 urllib3-2.8.0
```

```python
import requests

# Pedimos al usuario la especie que quiere buscar.
especie = input("Introduce una especie: ")

# URL de la API de personajes.
url = "https://rickandmortyapi.com/api/character"

# Realizamos una petición GET enviando la especie como filtro.
respuesta = requests.get(url, params={"species": especie})

# Comprobamos si la petición se ha realizado correctamente.
if respuesta.status_code == 200:

    # Convertimos la respuesta JSON en un diccionario de Python.
    datos = respuesta.json()

    # Obtenemos la lista de personajes.
    personajes = datos["results"]

    # Mostramos los nombres de los personajes encontrados.
    print("\nPersonajes de la especie", especie + ":")
```

```
    for personaje in personajes:
        print(personaje["name"])

else:
    print("No se han encontrado personajes de esa especie.")
```

Podemos observar cómo funcionan correctamente con distintas especies:

```
PS C:\Users\Nacho\Desktop\SGE> py UD00_4.13.py
Introduce una especie: Human

Personajes de la especie Human:
Rick Sanchez
Morty Smith
Summer Smith
Beth Smith
Jerry Smith
Abradolf Lincler
Adjudicator Rick
Agency Director
Alan Rails
Albert Einstein
Alexander
Annie
Antenna Morty
Antenna Rick
Ants in my Eyes Johnson
Aqua Morty
Aqua Rick
Artist Morty
Attila Starwar
Baby Legs
```

```
PS C:\Users\Nacho\Desktop\SGE> py UD00_4.13.py
Introduce una especie: Alien

Personajes de la especie Alien:
Abadango Cluster Princess
Alien Googah
Alien Morty
Alien Rick
Amish Cyborg
Arcade Alien
Armagheadon
Arthricia
Baby Wizard
Bearded Lady
Beebo
Bepisian
Beta-Seven
Birdperson
Blamph
Blim Blam
Blue Diplomat
Boobloosian
Borpocian
Canklanker Thom
```

4.14 PREGUNTA ENTREVISTA 14

El problema de les 8 Reines és àmpliament conegut
https://es.wikipedia.org/wiki/problema_de_las_ocho_reinas
Anem a donar-li una volta i anem a proposar el problema de les N reines.
Volem que faces un programa, que per a un nombre N (4 <= N <= 10) calcule de quantes formes diferents es poden col·locar N reines en un tauler de NxN dimensions.
NOTA IMPORTANT: no s'admeten precàlculs, el programa ha de calcular les solucions.
Per provar, els resultats haurien de ser:
Tamany N Solucions diferents Solucions totals (sense eliminar repeticions)
4 1 2
5 2 10
6 1 4
7 6 40
8 12 92
9 46 352
10 92 724
11 341 2,680
12 1,787 14,200
Per simplificar el problema, el vostre programa només haurà de calcular les "Solucions totals" del problema de les N reines (sense eliminar repeticions).

```
# Esta función comprueba si podemos colocar una reina en una
# determinada fila y columna sin que ataque a ninguna reina
# que ya hemos colocado anteriormente.
def esSeguro(tablero, fila, columna):

   # Recorremos todas las filas anteriores, donde ya tenemos
   # colocadas otras reinas.
   for i in range(fila):

       # Comprobamos si alguna reina está en la misma columna.
       # Si coinciden las columnas, las dos reinas se atacarían.
       if tablero[i] == columna:
           return False

       # Comprobamos si las dos reinas están en la misma diagonal.
       # Para estar en la misma diagonal, la diferencia entre
       # las filas debe ser igual a la diferencia entre las columnas.
       if abs(tablero[i] - columna) == abs(i - fila):
           return False

   # Si ninguna reina anterior puede atacar a la nueva reina,
   # significa que podemos colocarla en esa posición.
```

```
    return True


# Esta función utiliza recursividad para probar diferentes
# posiciones de las reinas y calcular todas las soluciones posibles.
def colocarReinas(tablero, fila, n):

    # Si la fila actual es igual a N significa que hemos conseguido
    # colocar una reina en todas las filas del tablero.
    # Por tanto, hemos encontrado una solución completa.
    if fila == n:
        return 1

    # Guardamos aquí el número de soluciones que vamos encontrando
    # mientras probamos las diferentes posiciones.
    soluciones = 0

    # Probamos todas las columnas posibles de la fila actual.
    for columna in range(n):

        # Antes de colocar la reina comprobamos que la posición
        # sea válida y que no ataque a ninguna reina anterior.
        if esSeguro(tablero, fila, columna):

            # Guardamos la columna donde hemos colocado la reina.
            # La posición de la lista representa la fila.
            tablero[fila] = columna

            # Pasamos a la siguiente fila y seguimos buscando
            # posibles posiciones para colocar otra reina.
            soluciones += colocarReinas(tablero, fila + 1, n)

            # Cuando terminamos de probar esta posición,
            # quitamos la reina para poder probar otra columna.
            # Esto es lo que permite volver atrás y buscar
            # diferentes soluciones.
            tablero[fila] = -1

    # Devolvemos todas las soluciones encontradas desde esta fila.
    return soluciones


# Pedimos al usuario el tamaño N del tablero.
```

```python
# El ejercicio establece que N debe estar entre 4 y 10.
n = int(input("Introduce el tamaño N (4-10): "))



# Comprobamos que el número introducido esté dentro del rango
# permitido por el ejercicio.
if n < 4 or n > 10:

   print("N debe estar entre 4 y 10.")

else:
   # Creamos una lista que representa el tablero.
   # Cada posición de la lista representa una fila.
   # Inicialmente ponemos -1 porque todavía no hay ninguna reina.
   #
   # Por ejemplo, para N = 4:
   # [-1, -1, -1, -1]
   tablero = [-1] * n

   # Comenzamos a colocar las reinas desde la fila 0.
   # La función irá probando todas las posiciones posibles
   # y contará las soluciones que encuentre.
   soluciones = colocarReinas(tablero, 0, n)

   # Mostramos por pantalla el número total de soluciones.
   print("Soluciones totales:", soluciones)
```

```
ignglltar@ORJ1:~/Escritorio/Python$ python3 UD00_4.14.py
Introduce el tamaño N (4-10): 8
Soluciones totales: 92
```

4.15 PREGUNTA ENTREVISTA 15

Fes un programa que llegisca d'un fitxer que es passe com a paràmetre, un llistat de paraules (una paraula per línia). Aquestes paraules serà un llistat de "paraules prohibides".
Utilitzant la biblioteca https://pyperclip.readthedocs.io/en/latest/, realitzeu un programa que quan copies alguna cosa del porta-retalls, analitze el text copiat i substituïsca de les paraules prohibides (sense distingir majúscules i minúscules) els caràcters de les paraules prohibides al llistat per * i el torne a copiar al porta-retalls.
Exemple de funcionament:
Cridem al programa "programa.py llista.txt"
El fitxer "llista.txt" content les paraules:
Hola

pYThon
Amb el programa en marxa, si copie al porta-retalls la frase “HOLA Sergi viva PYTHON”, hauria de modificar al porta-retalls aquesta frase i deixar-la com “** Sergi viva ******”, de forma que quan peguem, es pegue aquesta segona versió.**

```
import sys
import time
import pyperclip


# Comprobamos que se haya pasado el archivo como parámetro
if len(sys.argv) != 2:
   print("Uso: python programa.py llista.txt")
   sys.exit()


# Nombre del archivo recibido
nombre_archivo = sys.argv[1]

# Leemos las palabras prohibidas
with open(nombre_archivo, "r", encoding="utf-8") as archivo:
   palabras_prohibidas = []

   for linea in archivo:
       palabra = linea.strip()

       if palabra:
           palabras_prohibidas.append(palabra)


# Guardamos el contenido actual del portapapeles
texto_anterior = pyperclip.paste()

print("Programa en marcha...")
print("Copia un texto para comprobarlo.")


# Comprobamos continuamente el portapapeles
while True:

   texto_actual = pyperclip.paste()

   # Solo analizamos el texto si ha cambiado
```

```
    if texto_actual != texto_anterior:

        texto_modificado = texto_actual

        # Comprobamos cada palabra prohibida
        for palabra in palabras_prohibidas:

            # Creamos una versión en minúsculas para comparar
            texto_minusculas = texto_modificado.lower()
            palabra_minusculas = palabra.lower()

            # Buscamos la palabra sin distinguir mayúsculas/minúsculas
            while palabra_minusculas in texto_minusculas:

                posicion = texto_minusculas.find(palabra_minusculas)

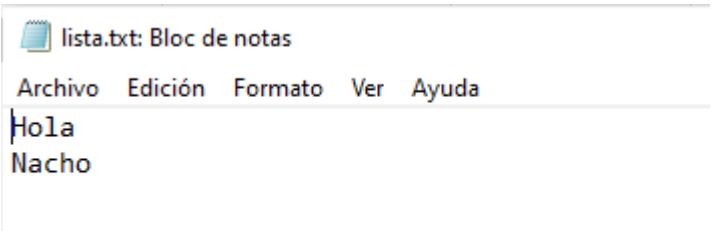
                   # Sustituimos la palabra por tantos * como caracteres
tenga
                texto_modificado = (
                    texto_modificado[:posicion]
                    + "*" * len(palabra)
                    + texto_modificado[posicion + len(palabra):]
                )

                # Actualizamos la versión en minúsculas
                texto_minusculas = texto_modificado.lower()

        # Si hemos modificado el texto, lo copiamos al portapapeles
        if texto_modificado != texto_actual:
            pyperclip.copy(texto_modificado)
            print("Texto modificado:", texto_modificado)

        texto_anterior = texto_modificado

    # Esperamos un poco antes de volver a comprobar
    time.sleep(0.2)
```



Aquí podemos comprobar que si copio Hola me llamo Nacho funciona correctamente

```
(.venv) PS C:\Users\Nacho\Desktop\SGE\UD00_4.15> py UD00_4.15.py lista.txt
Programa en marcha...
Copia un texto para comprobarlo.
Texto modificado: **** me llamo *****
```